Proyecto 2
Algoritmos BFS y DFS

Josue gutierrez Andrade

Diseño y Análisis de Algoritmos

Resumen

Sobre la biblioteca de grafos del Proyecto 1 se implementan tres métodos de la clase `Grafo`: `BFS`, `DFS_R` y `DFS_I`. Dado un nodo fuente s, cada uno calcula el árbol inducido por la búsqueda a lo ancho o por la búsqueda en profundidad (recursiva e iterativa). Los métodos se aplicaron a 18 grafos, generados con seis modelos y con 30, 100 y 500 nodos, lo que dio 54 árboles. Todos los grafos y árboles se exportaron a formato GraphViz y se visualizaron.

Índice

1. Introducción **2**

2. Representación de los grafos **2**

3. Algoritmos **2**

3.1. Búsqueda a lo ancho: `BFS(self, s)` 2

3.2. Búsqueda en profundidad recursiva: `DFS_R(self, s)` 3

3.3. Búsqueda en profundidad iterativa: `DFS_I(self, s)` 3

3.4. Verificación 4

4. Experimentos **4**

5. Resultados **5**

6. Visualización **7**

1. Introducción

La búsqueda a lo ancho (BFS) y la búsqueda en profundidad (DFS) recorren todos los nodos alcanzables desde un nodo fuente s. Cada recorrido define un árbol con raíz en s: cuando un nodo v se descubre desde un nodo u, la arista (u, v) pasa a formar parte del árbol y u se convierte en el padre de v. Los dos recorridos producen árboles con formas muy distintas. BFS genera árboles anchos y poco profundos, cuyos niveles coinciden con las distancias a s. DFS genera árboles angostos y profundos, que siguen un camino tan lejos como pueden antes de regresar.

2. Representación de los grafos

Los métodos se agregaron a la clase `Grafo` del Proyecto 1. Esa clase almacena el grafo como una lista de adyacencia basada en conjuntos: el atributo `_vecinos` asocia a cada identificador de nodo el conjunto de identificadores adyacentes. Con esta representación, recorrer los vecinos de un nodo u cuesta $O(\text{grado}(u))$ y consultar si un nodo ya se visitó cuesta $O(1)$ en promedio, de modo que los tres recorridos toman tiempo $O(n + m)$, con n nodos y m aristas.

Cada método devuelve el árbol como un nuevo objeto `Grafo`, con el mismo tipo (dirigido o no) que el original. El árbol contiene los nodos alcanzables desde s y las aristas (padre, hijo) del recorrido. Por eso puede exportarse con `guardar_graphviz`, igual que cualquier otro grafo. Si s no existe en el grafo, los tres métodos lanzan `KeyError`.

3. Algoritmos

3.1. Búsqueda a lo ancho: `BFS(self, s)`

Se sigue el algoritmo por capas visto en clase:

$$L_0 = \{s\}, \qquad L_{i+1} = \{\, v \notin L_0 \cup \cdots \cup L_i \;:\; (u, v) \in E \text{ para algún } u \in L_i \,\}.$$

Se recorre cada nodo u de la capa actual y, por cada vecino v que aún no se ha descubierto, se marca v como descubierto, se agrega la arista (u, v) al árbol y v se añade a la siguiente capa. El ciclo termina cuando una capa queda vacía.

```
def BFS(self, s):
    self._validar_fuente(s)
    arbol = Grafo(self.dirigido)
    arbol.agregar_nodo(s)
    descubierto = {s}
    capa = [s]
    while capa:
        siguiente_capa = []
        for u in capa:
            for v in self._vecinos[u]:
                if v not in descubierto:
                    descubierto.add(v)
                    arbol.agregar_arista(u, v)
                    siguiente_capa.append(v)
        capa = siguiente_capa
    return arbol
```

Código 1: Método `BFS`.

Cada nodo entra a lo más una vez a una capa y cada lista de vecinos se recorre una sola vez, así que el costo es $O(n+m)$. El nodo v queda en la capa L_j exactamente cuando su distancia a s es j, y su padre está en L_{j-1}. En consecuencia, los extremos de toda arista del árbol BFS están en niveles consecutivos, y la altura del árbol es la distancia máxima desde s.

3.2. Búsqueda en profundidad recursiva: `DFS_R(self, s)`

Es el algoritmo DFS(G, u) visto en clase: se marca u como explorado y, por cada arista (u, v) con v no explorado, se agrega (u, v) al árbol y se invoca recursivamente DFS(G, v).

```
def DFS_R(self, s):
    self._validar_fuente(s)
    arbol = Grafo(self.dirigido)
    arbol.agregar_nodo(s)
    # La profundidad de la recursión puede llegar al número de nodos
    limite_original = sys.getrecursionlimit()
    sys.setrecursionlimit(max(limite_original, len(self.nodos) + 100))
    try:
        self._dfs_recursivo(s, set(), arbol)
    finally:
        sys.setrecursionlimit(limite_original)
    return arbol

def _dfs_recursivo(self, u, explorado, arbol):
    explorado.add(u)
    for v in self._vecinos[u]:
        if v not in explorado:
            arbol.agregar_arista(u, v)
            self._dfs_recursivo(v, explorado, arbol)
```

Código 2: Métodos `DFS_R` y `_dfs_recursivo`.

En el peor caso la recursión alcanza una profundidad igual al número de nodos (por ejemplo, en un camino). Python limita por omisión la profundidad a 1000 llamadas, por lo que `DFS_R` aumenta el límite antes de empezar y lo restaura al terminar. Aun así, la pila del sistema operativo impone un tope: en Python 3.6 sobre Windows, la recursión funciona hasta unos 4000 niveles de profundidad. Los árboles de este proyecto llegan a lo más a 417 niveles, muy por debajo de ese tope. El costo es $O(n+m)$.

3.3. Búsqueda en profundidad iterativa: `DFS_I(self, s)`

La versión iterativa sustituye la pila de llamadas de Python por una pila explícita. Cada elemento de la pila es un par $(u, \text{pendientes})$, donde *pendientes* es un iterador sobre los vecinos de u que aún no se han revisado. En cada paso se toma el elemento del tope y se avanza su iterador hasta encontrar un vecino v no explorado. Si lo encuentra, agrega (u, v) al árbol y apila v, lo que equivale a la llamada recursiva. Si el iterador se agota, u se desapila, lo que equivale al regreso de la llamada.

```
def DFS_I(self, s):
    self._validar_fuente(s)
    arbol = Grafo(self.dirigido)
    arbol.agregar_nodo(s)
    explorado = {s}
    pila = [(s, iter(self._vecinos[s]))]
    while pila:
```

```
        u, pendientes = pila[-1]
        for v in pendientes:
            if v not in explorado:
                explorado.add(v)
                arbol.agregar_arista(u, v)
                pila.append((v, iter(self._vecinos[v])))
                break
        else:
            # u ya no tiene vecinos sin explorar: se regresa a su padre
            pila.pop()
    return arbol
```

Código 3: Método `DFS_I`.

Como el iterador conserva la posición en la lista de vecinos, cada vecino se revisa una sola vez y el costo es $O(n+m)$. Además, la pila contiene en todo momento los mismos nodos que la pila de llamadas de `DFS_R`, y ambos métodos revisan los vecinos en el mismo orden. Por lo tanto, `DFS_I` y `DFS_R` producen exactamente el mismo árbol a partir del mismo nodo fuente. A diferencia de la versión recursiva, `DFS_I` no depende del límite de recursión de Python.

3.4. Verificación

Además de generar los archivos, se comprobó con un programa de prueba que, en grafos de los seis modelos y de los tres tamaños, se cumplen las siguientes propiedades:

- Los tres árboles contienen todos los nodos alcanzables desde s, tienen $n-1$ aristas, son conexos y solo usan aristas del grafo original.
- El nivel de cada nodo en el árbol BFS es igual a su distancia mínima a s, calculada de forma independiente.
- En los árboles DFS, toda arista del grafo que no pertenece al árbol une a un nodo con uno de sus ancestros; es decir, no hay aristas cruzadas.
- `DFS_R` y `DFS_I` producen la misma lista de aristas.
- En un grafo no conexo, el árbol solo contiene la componente de s.
- `DFS_R` procesa un camino de 4000 nodos sin exceder el límite de recursión.

Todo el código, incluidos los scripts de generación y de imágenes, se ejecutó con Python 3.6.8, la versión solicitada.

4. Experimentos

El script `generar_gv.py` genera tres grafos por cada uno de los seis modelos del Proyecto 1, con 30, 100 y 500 nodos. Cada grafo se vuelve a generar hasta que resulta conexo, de modo que los árboles abarcan todos sus nodos. Luego calcula los tres árboles de cada grafo desde el nodo fuente s, que es el primer nodo del grafo: 0 en los modelos aleatorios y $(0, 0)$, una esquina, en la malla.

Modelo	Parámetros
Erdős–Rényi	$m = \text{round}\left(\dfrac{n(\ln n + 2)}{2}\right)$
Gilbert	$p = \dfrac{\ln n + 3}{n}$
Geográfico simple	$r = \sqrt{\dfrac{\ln n + 4}{\pi n}}$
Barabási–Albert	$d = 4$
Dorogovtsev–Mendes	sin parámetros adicionales
Malla	5×6, 10×10 y 20×25

Tabla 1: Parámetros de los generadores.

Los grafos se guardan en `gv/generados/` y los árboles en `gv/calculados/`. El script `generar_imagenes.py` dibuja todos los archivos con Graphviz y elige la forma de dibujo según el tipo de grafo:

- **Árboles BFS:** `twopi`, que coloca la raíz al centro y cada capa en un anillo.
- **Malla y sus árboles:** `neato`, con cada nodo fijo en su posición de la cuadrícula.
- **Demás grafos y árboles DFS:** `sfdp`.

5. Resultados

La tabla 2 muestra el tamaño de cada grafo generado y la altura de sus árboles, es decir, el nivel más profundo alcanzado desde s. Los grafos son aleatorios, así que los valores corresponden a la ejecución incluida en el repositorio y cambian si los grafos se vuelven a generar.

			Altura del árbol	
Grafo	Nodos	Aristas	BFS	DFS
Erdős–Rényi	30	81	3	26
Gilbert	30	80	4	22
Geográfico simple	30	89	5	26
Barabási–Albert	30	55	9	18
Dorogovtsev–Mendes	30	57	3	7
Malla 5×6	30	49	9	22
Erdős–Rényi	100	330	5	83
Gilbert	100	367	4	82
Geográfico simple	100	368	9	65
Barabási–Albert	100	197	21	66
Dorogovtsev–Mendes	100	197	4	16
Malla 10×10	100	180	18	78
Erdős–Rényi	500	2054	5	406
Gilbert	500	2349	4	417
Geográfico simple	500	2348	17	338
Barabási–Albert	500	996	90	287
Dorogovtsev–Mendes	500	997	5	21
Malla 20×25	500	955	43	299

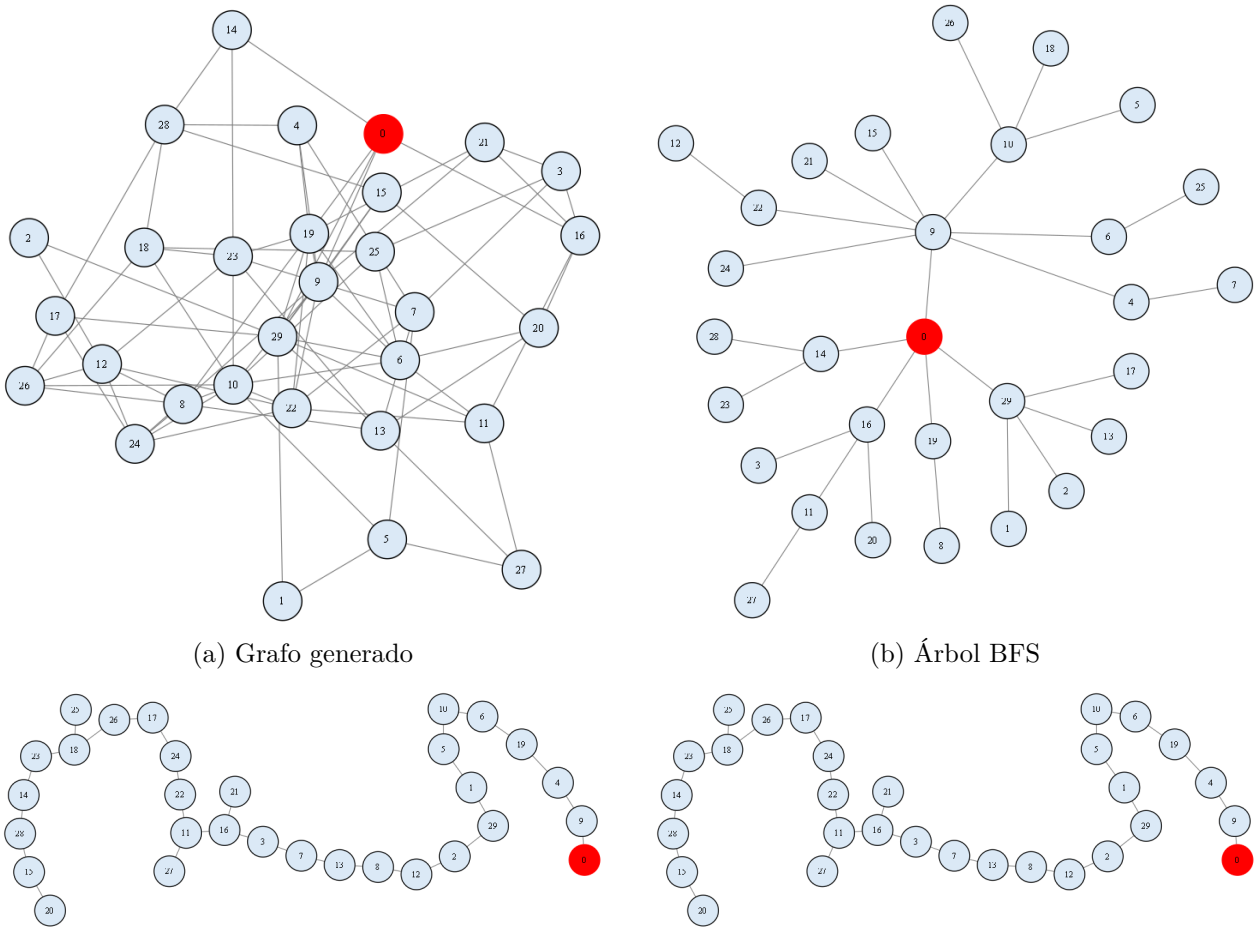
Tabla 2: Tamaño de los grafos generados y altura de sus árboles. La columna DFS vale para `DFS_R` y `DFS_I`, que producen el mismo árbol.

De la tabla se desprenden varias observaciones:

- **BFS produce árboles poco profundos.** En los modelos de Erdős–Rényi, Gilbert y Dorogovtsev–Mendes, la altura del árbol BFS se mantiene entre 3 y 5 aunque n pase de 30 a 500: todo nodo está a pocos pasos de s.
- **DFS produce árboles profundos.** En los grafos más densos, el árbol DFS es casi un camino: con 500 nodos, su altura supera 400 en Erdős–Rényi y Gilbert.
- **Dorogovtsev–Mendes es la excepción en DFS.** Su árbol DFS es mucho más bajo que en los demás modelos (21 con 500 nodos), porque cada nodo nuevo se conecta solo a los extremos de una arista existente, y los nodos antiguos concentran casi todas las conexiones.
- **La geometría limita a BFS.** En el modelo geográfico y en la malla, las aristas solo unen nodos cercanos, así que la distancia desde s crece con el tamaño del grafo. En la malla, la altura BFS es exactamente la distancia de la esquina $(0, 0)$ a la esquina opuesta: $(f-1)+(c-1)$ para una malla de $f \times c$.
- **Barabási–Albert tiene el árbol BFS más alto.** La variante implementada limita el grado de cada nodo a $d = 4$, por lo que no se forman nodos muy conectados que acorten las distancias.

6. Visualización

Las figuras 1 a 18 muestran cada grafo generado junto con sus tres árboles. Como `DFS_R` y `DFS_I` producen el mismo árbol, sus imágenes coinciden.



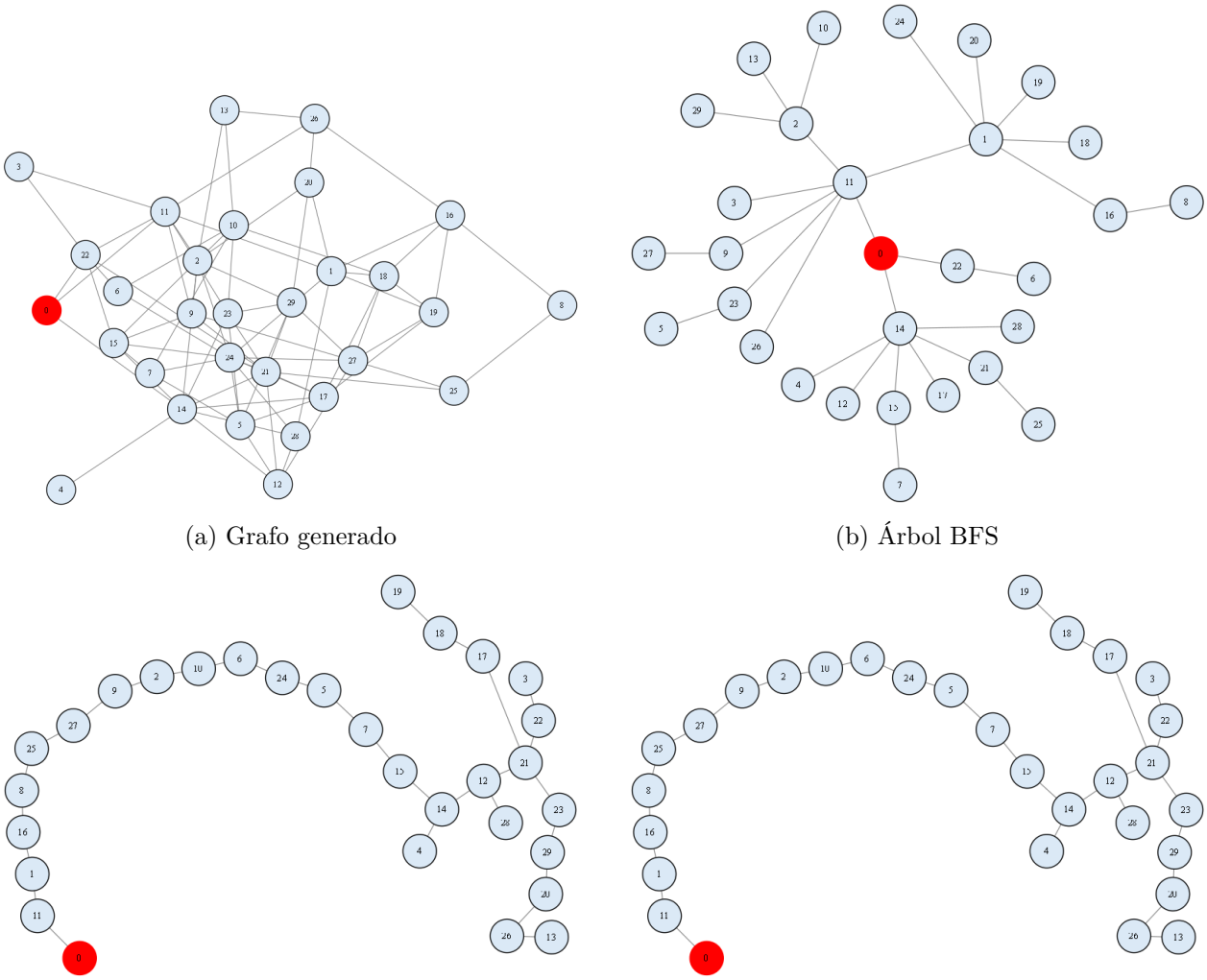
(a) Grafo generado

(b) Árbol BFS

(c) Árbol DFS recursivo

(d) Árbol DFS iterativo

Figura 1: Erdős–Rényi con 30 nodos. El nodo fuente aparece en rojo.



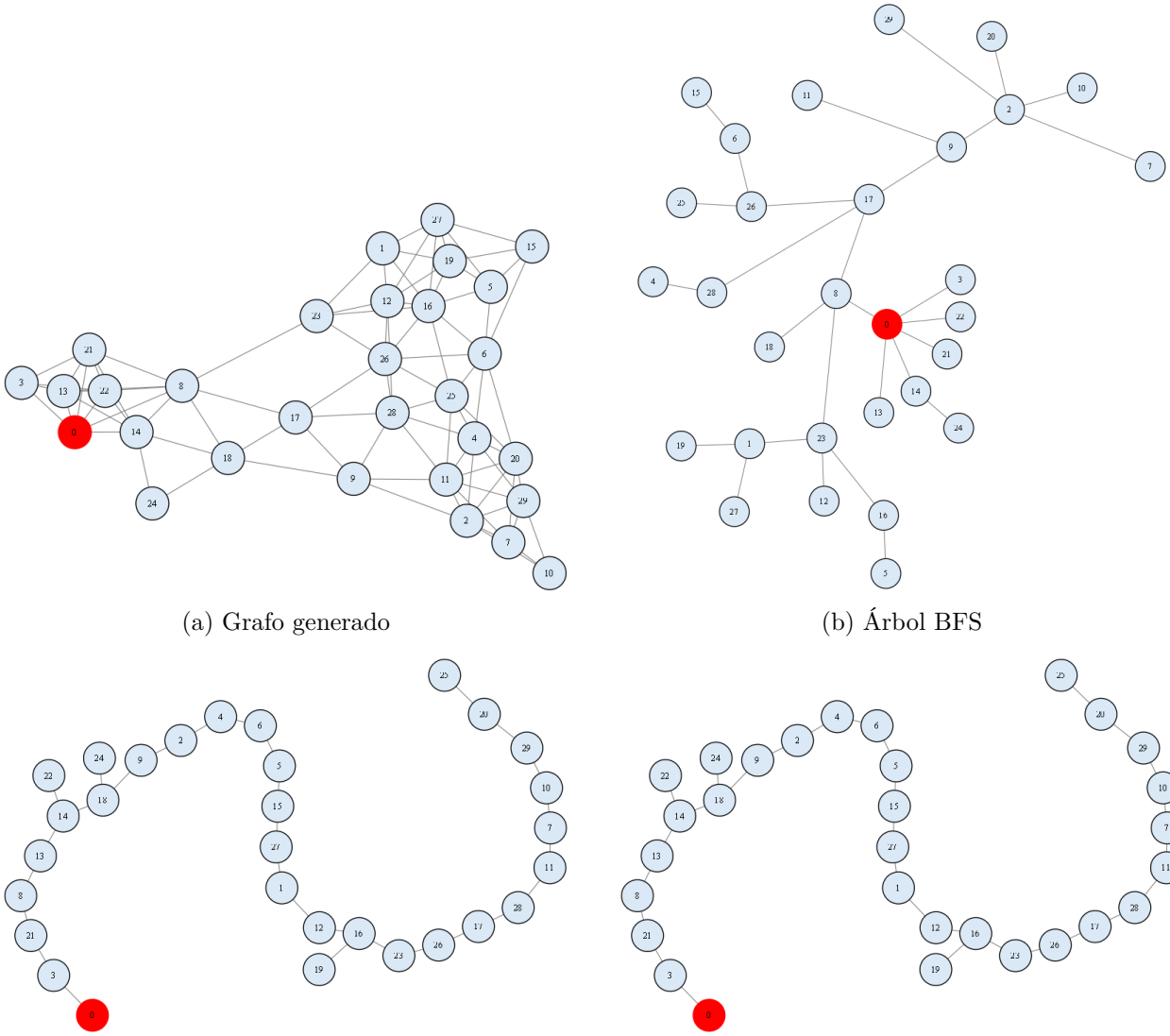
(a) Grafo generado

(b) Árbol BFS

(c) Árbol DFS recursivo

(d) Árbol DFS iterativo

Figura 2: Gilbert con 30 nodos. El nodo fuente aparece en rojo.



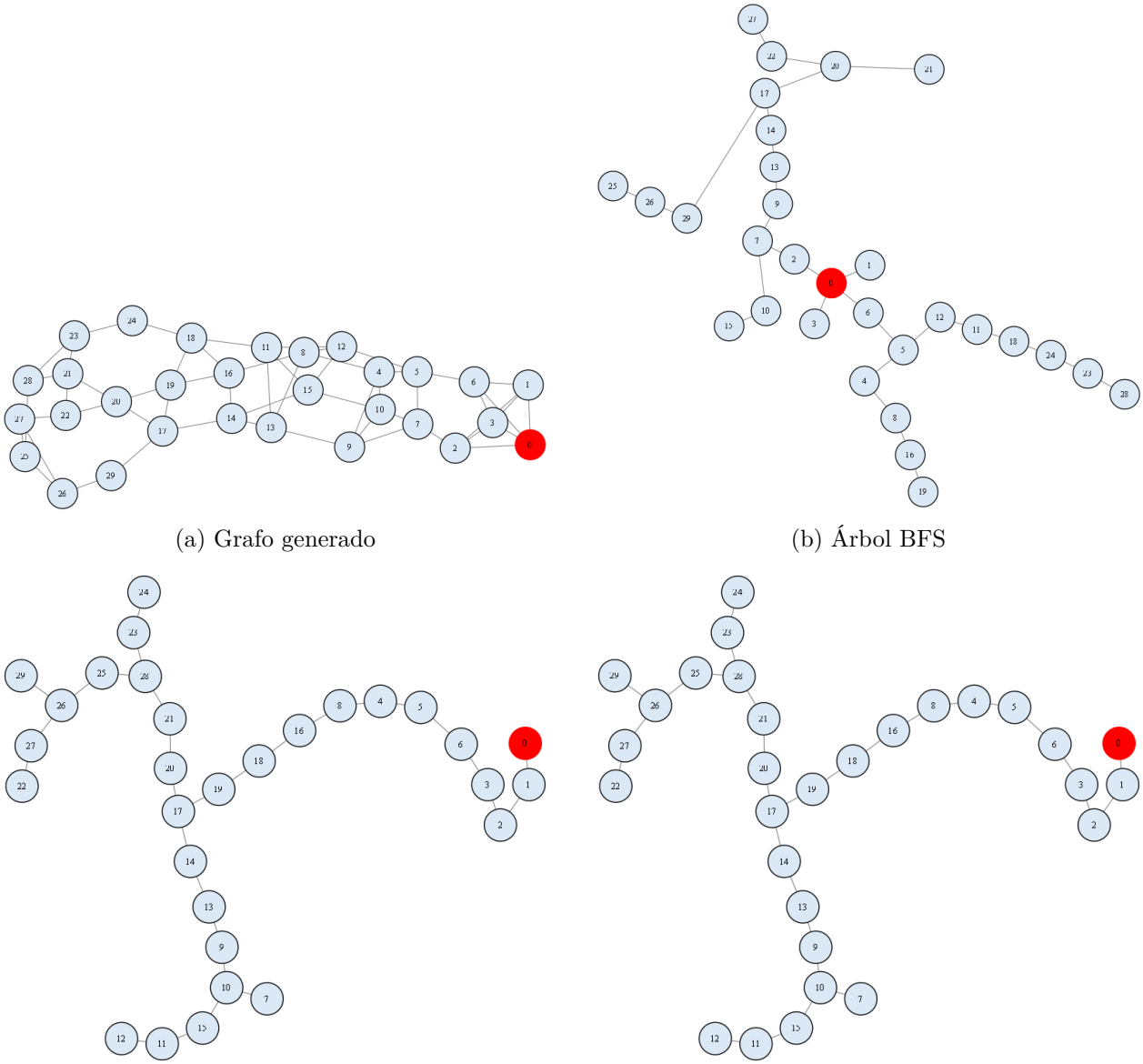
(a) Grafo generado

(b) Árbol BFS

(c) Árbol DFS recursivo

(d) Árbol DFS iterativo

Figura 3: Geográfico simple con 30 nodos. El nodo fuente aparece en rojo.



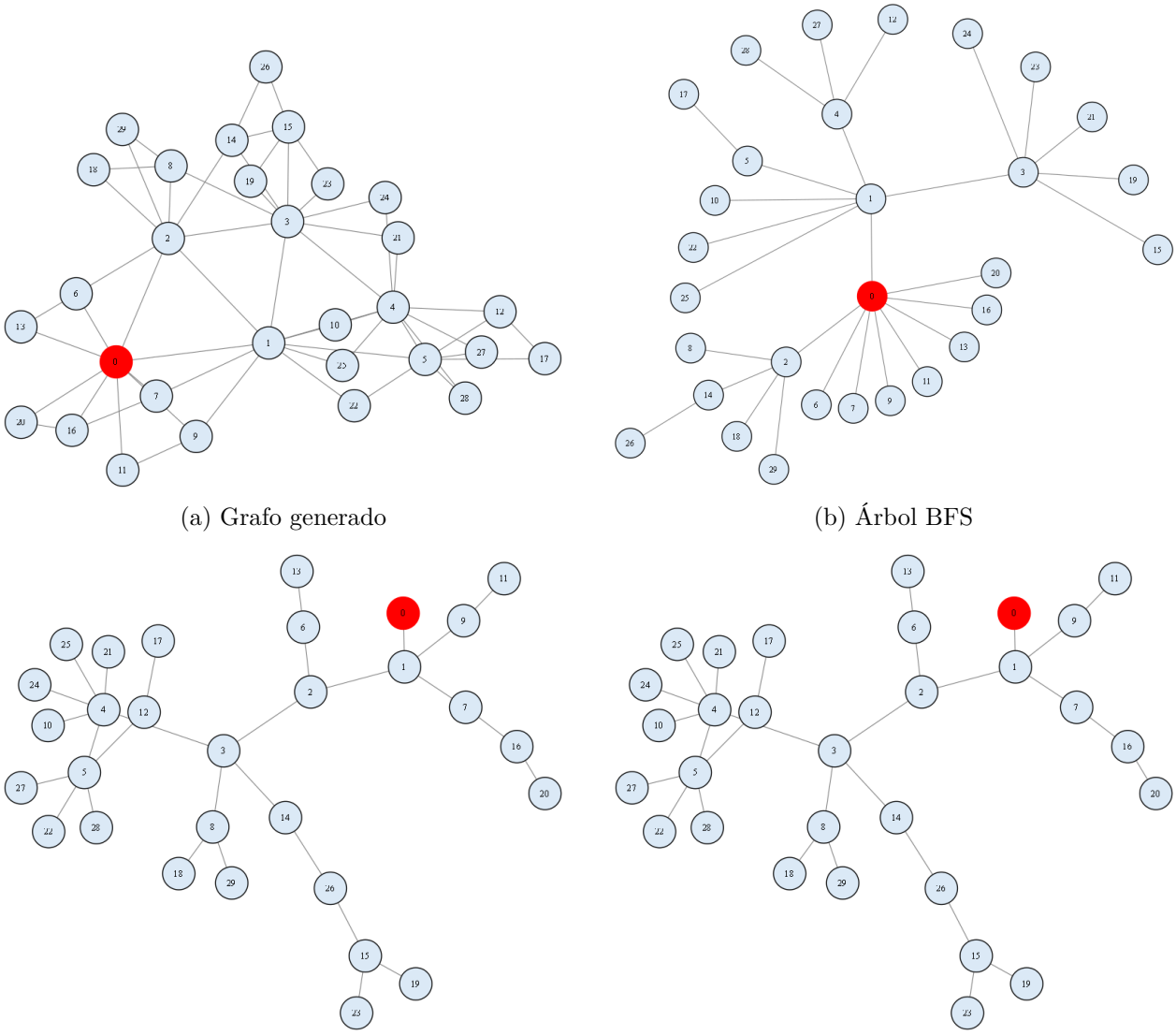
(a) Grafo generado

(b) Árbol BFS

(c) Árbol DFS recursivo

(d) Árbol DFS iterativo

Figura 4: Barabási–Albert con 30 nodos. El nodo fuente aparece en rojo.



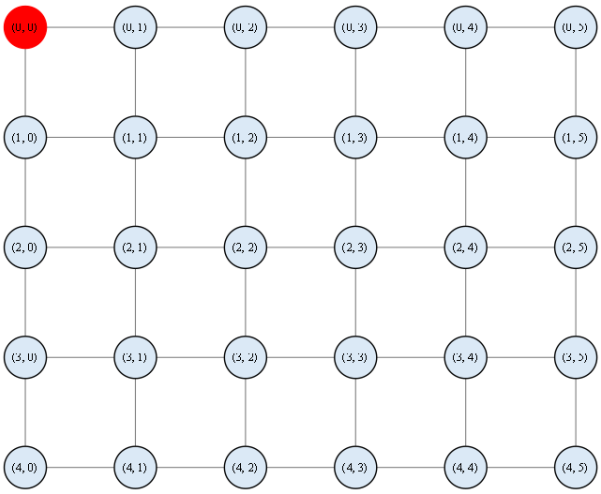
(a) Grafo generado

(b) Árbol BFS

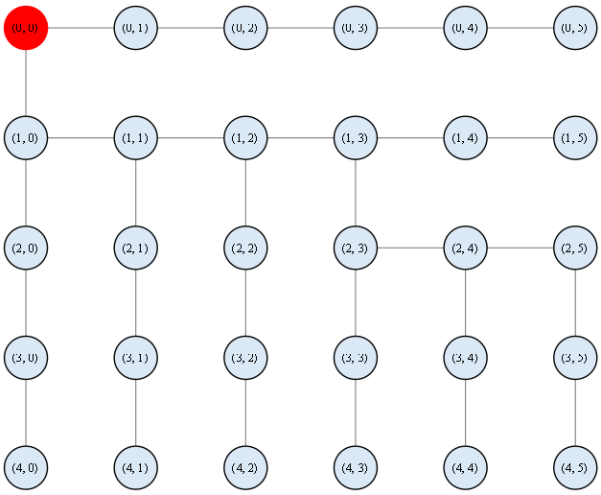
(c) Árbol DFS recursivo

(d) Árbol DFS iterativo

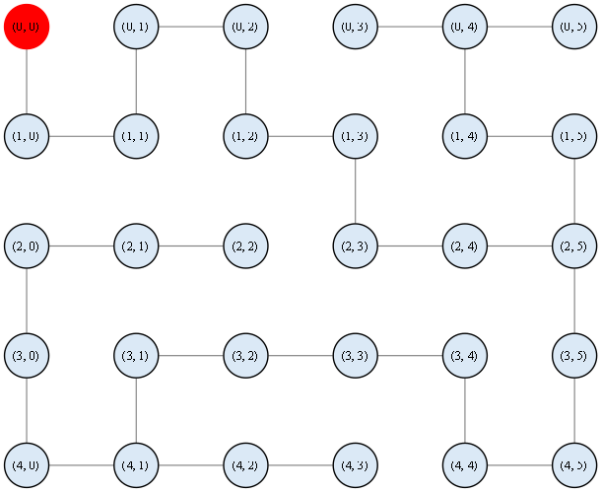
Figura 5: Dorogovtsev–Mendes con 30 nodos. El nodo fuente aparece en rojo.



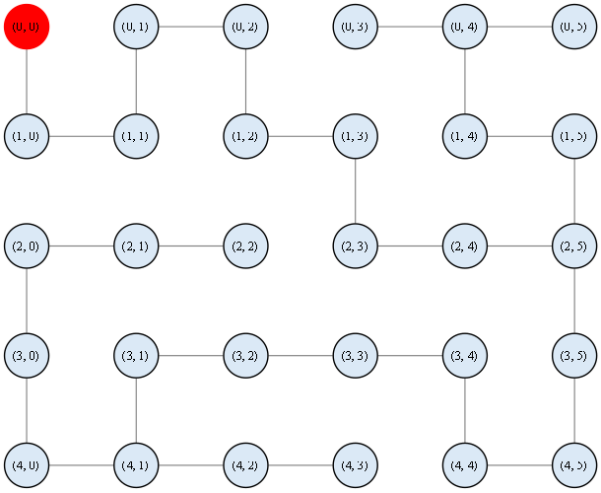
(a) Grafo generado



(b) Árbol BFS

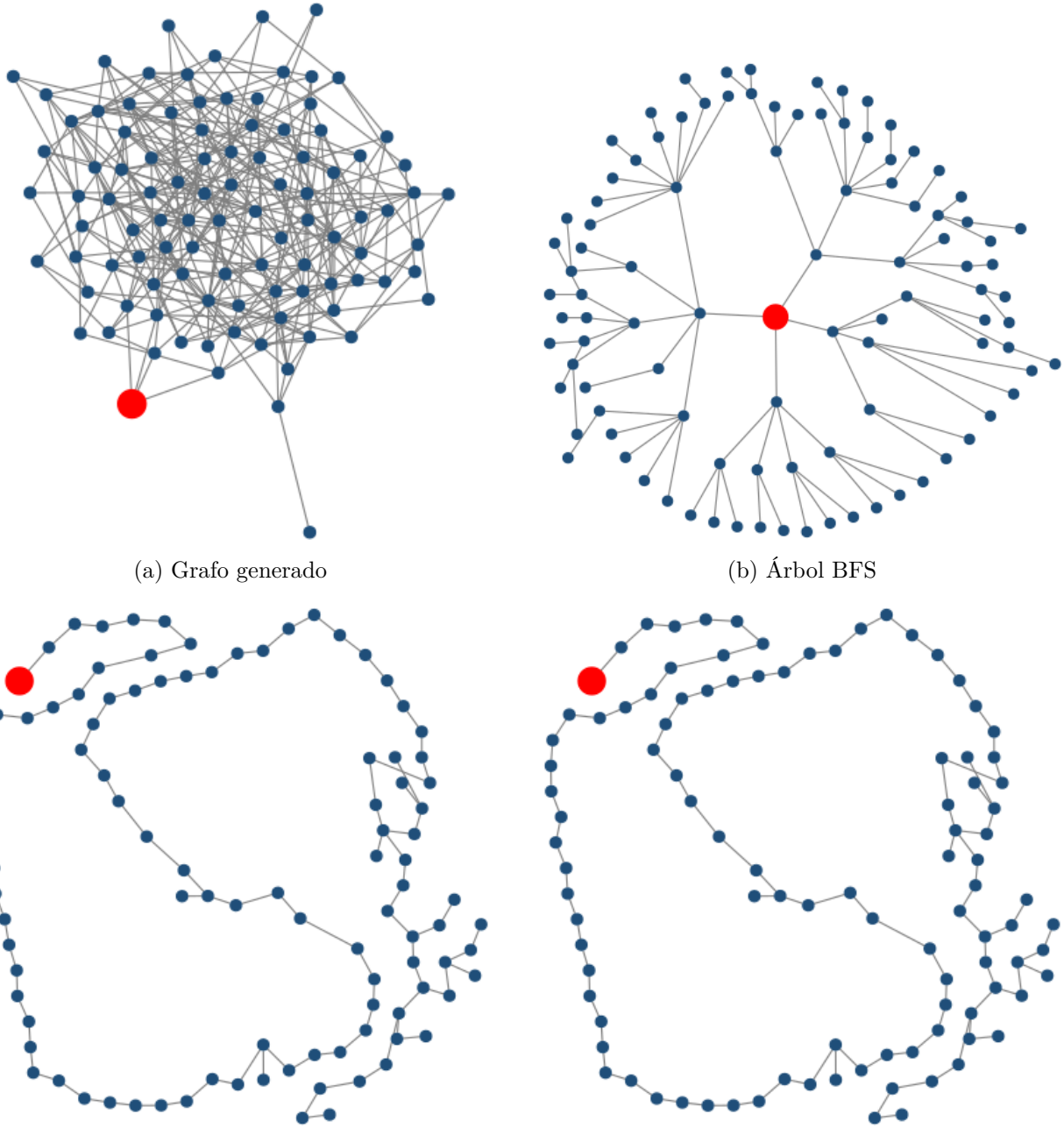


(c) Árbol DFS recursivo



(d) Árbol DFS iterativo

Figura 6: Malla de 5×6. El nodo fuente aparece en rojo.



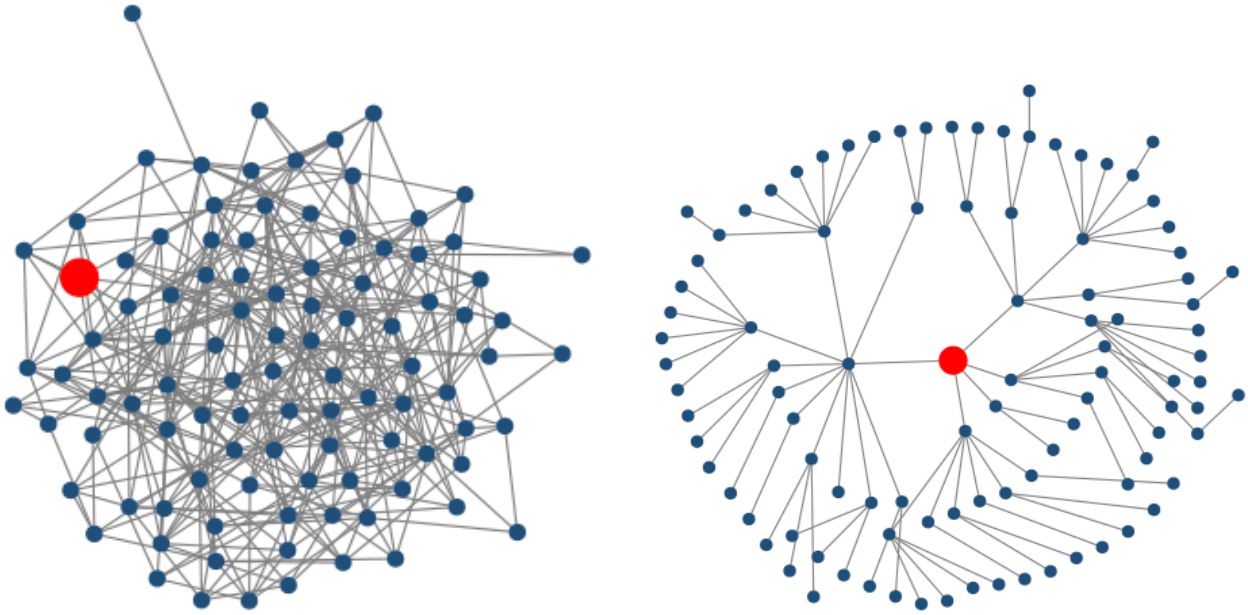
(a) Grafo generado

(b) Árbol BFS

(c) Árbol DFS recursivo

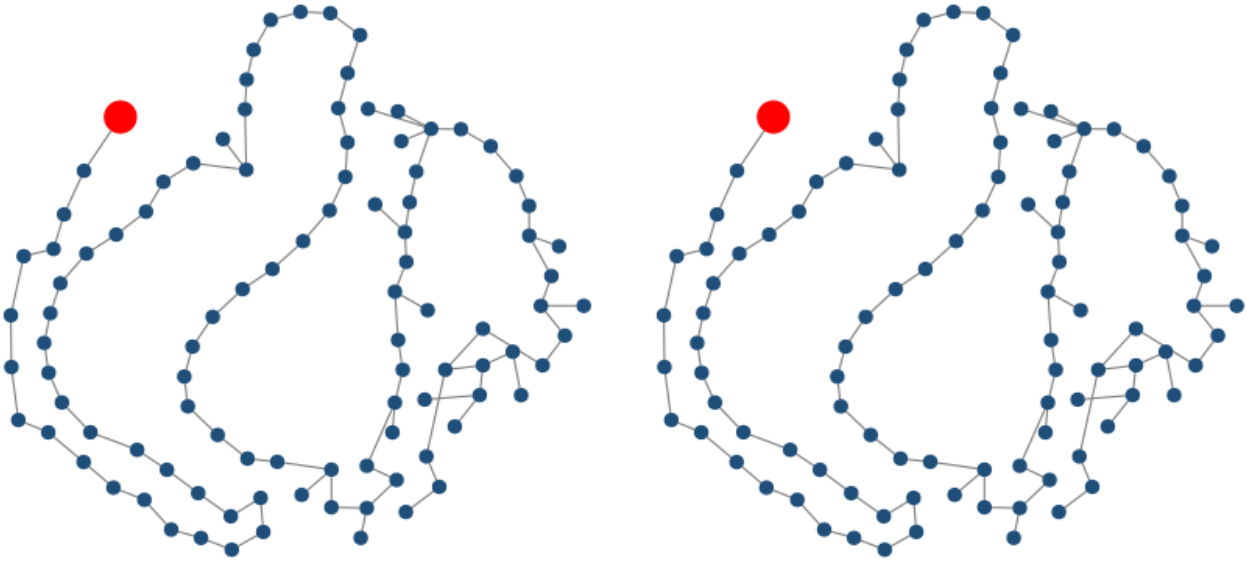
(d) Árbol DFS iterativo

Figura 7: Erdős–Rényi con 100 nodos. El nodo fuente aparece en rojo.



(a) Grafo generado

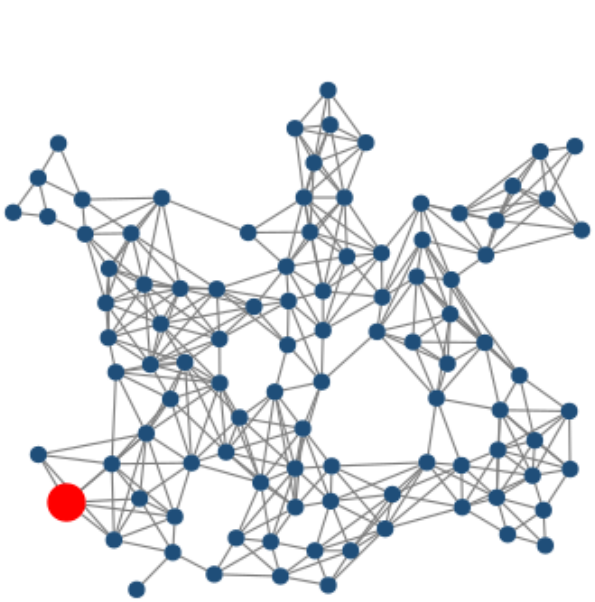
(b) Árbol BFS



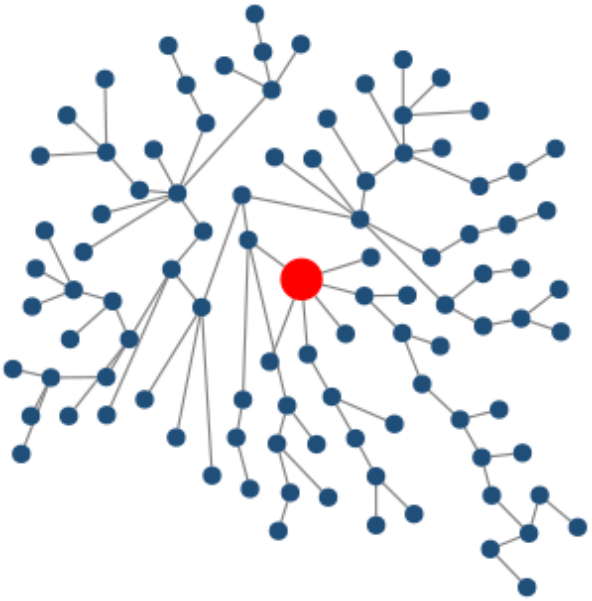
(c) Árbol DFS recursivo

(d) Árbol DFS iterativo

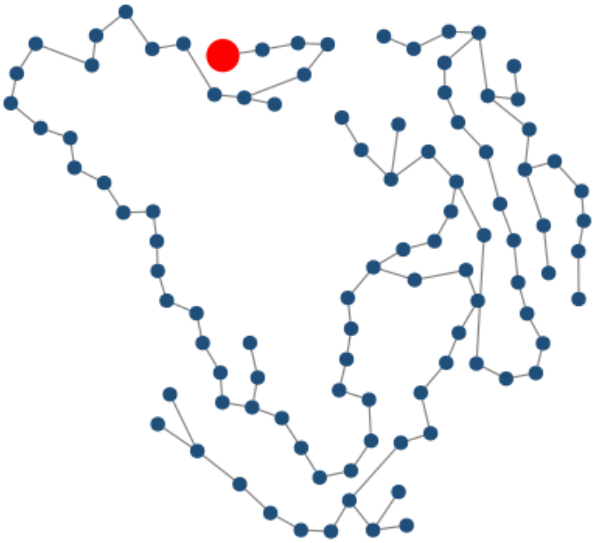
Figura 8: Gilbert con 100 nodos. El nodo fuente aparece en rojo.



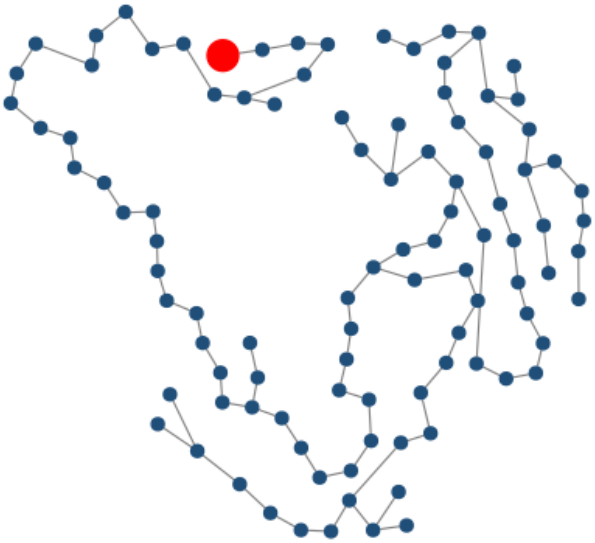
(a) Grafo generado



(b) Árbol BFS

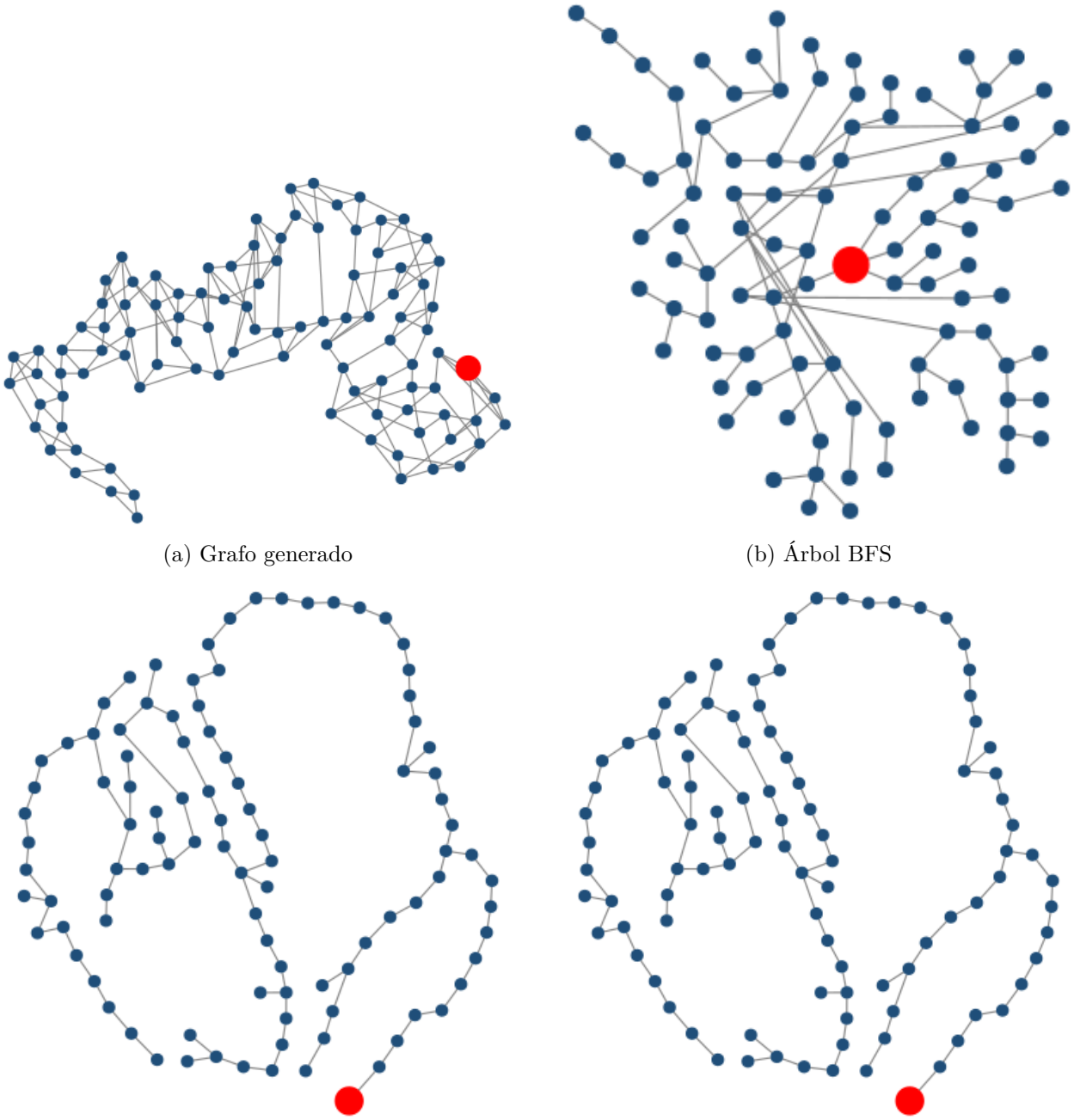


(c) Árbol DFS recursivo



(d) Árbol DFS iterativo

Figura 9: Geográfico simple con 100 nodos. El nodo fuente aparece en rojo.



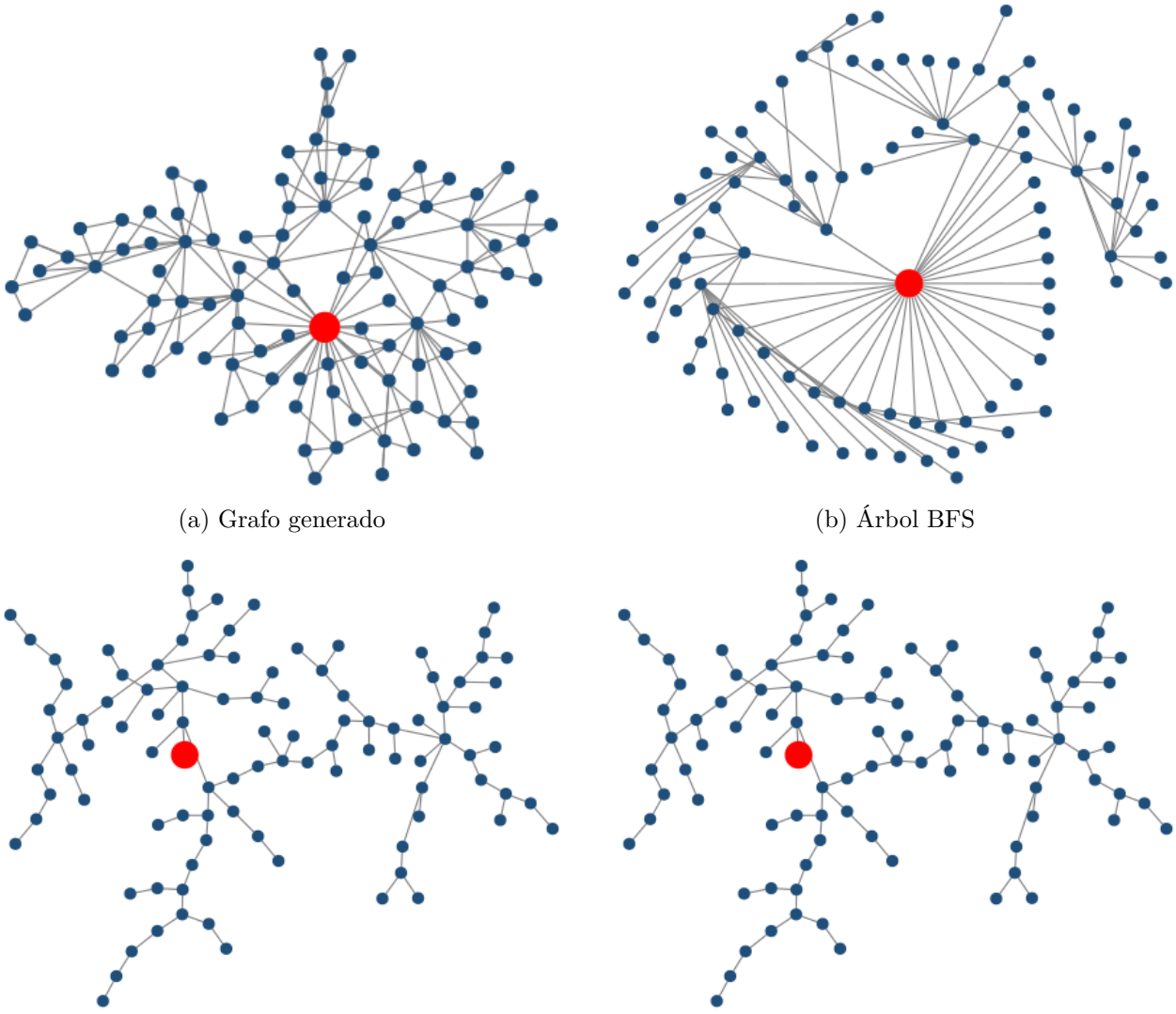
(a) Grafo generado

(b) Árbol BFS

(c) Árbol DFS recursivo

(d) Árbol DFS iterativo

Figura 10: Barabási–Albert con 100 nodos. El nodo fuente aparece en rojo.



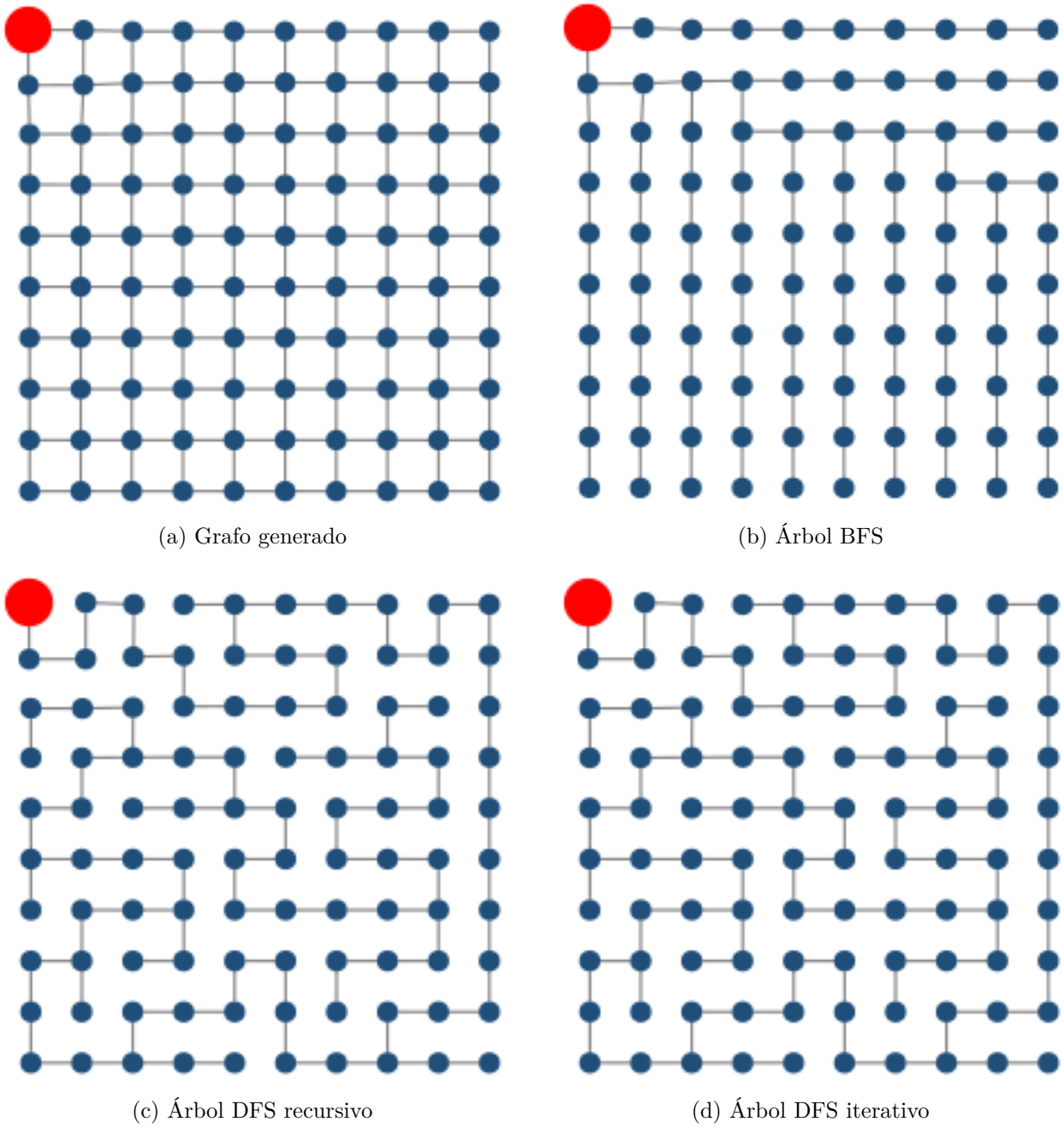
(a) Grafo generado

(b) Árbol BFS

(c) Árbol DFS recursivo

(d) Árbol DFS iterativo

Figura 11: Dorogovtsev–Mendes con 100 nodos. El nodo fuente aparece en rojo.



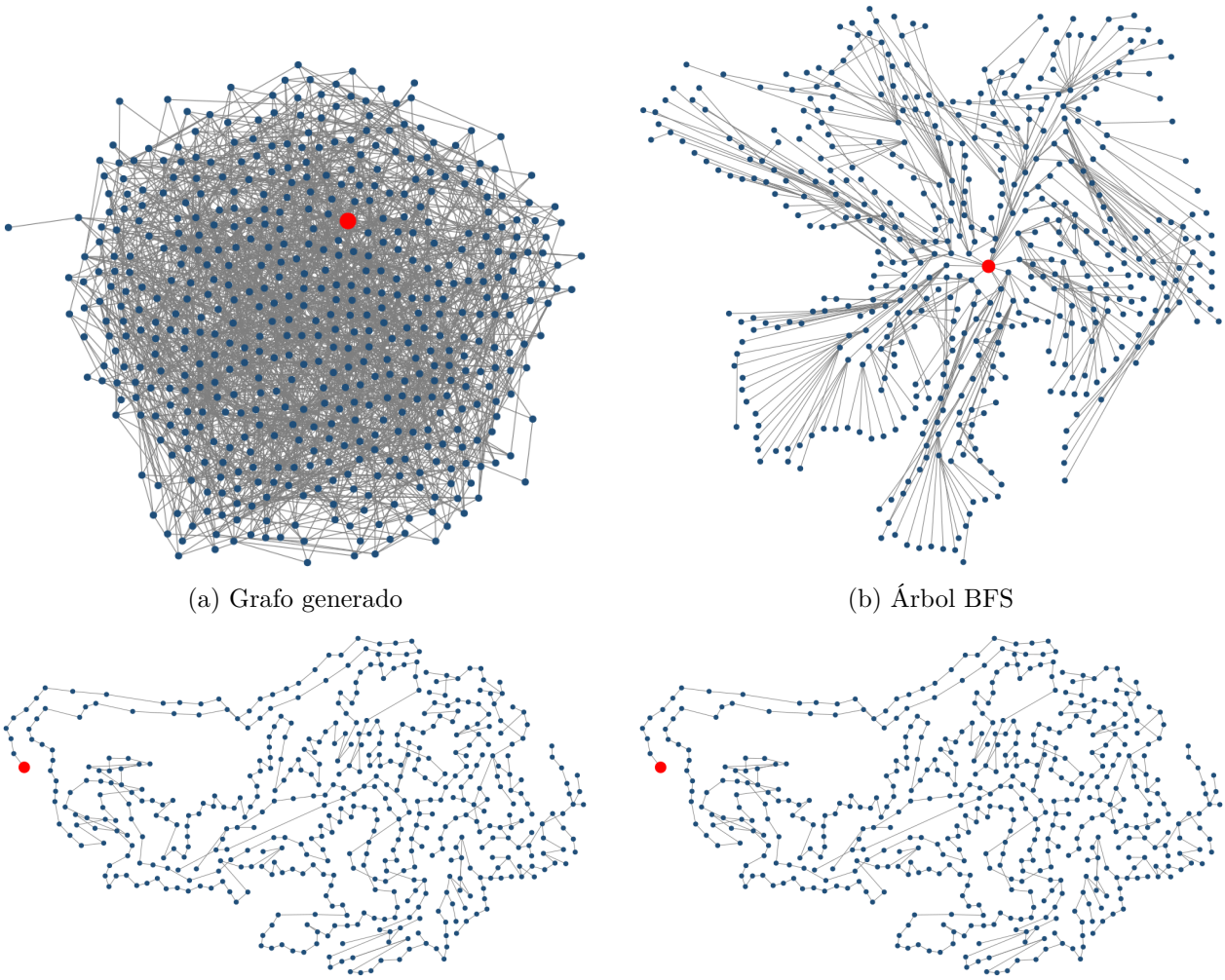
(a) Grafo generado

(b) Árbol BFS

(c) Árbol DFS recursivo

(d) Árbol DFS iterativo

Figura 12: Malla de 10×10. El nodo fuente aparece en rojo.



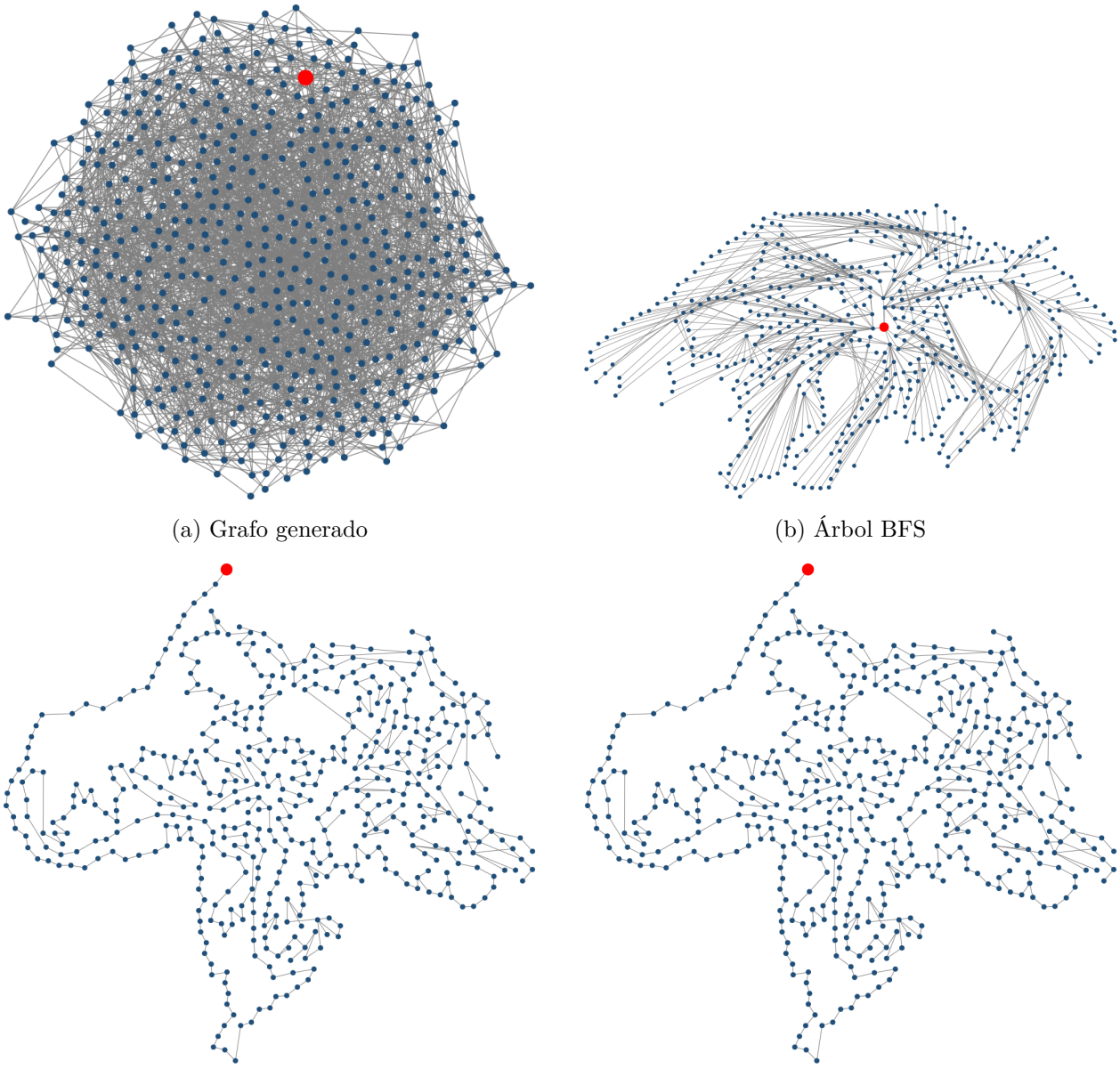
(a) Grafo generado

(b) Árbol BFS

(c) Árbol DFS recursivo

(d) Árbol DFS iterativo

Figura 13: Erdős–Rényi con 500 nodos. El nodo fuente aparece en rojo.



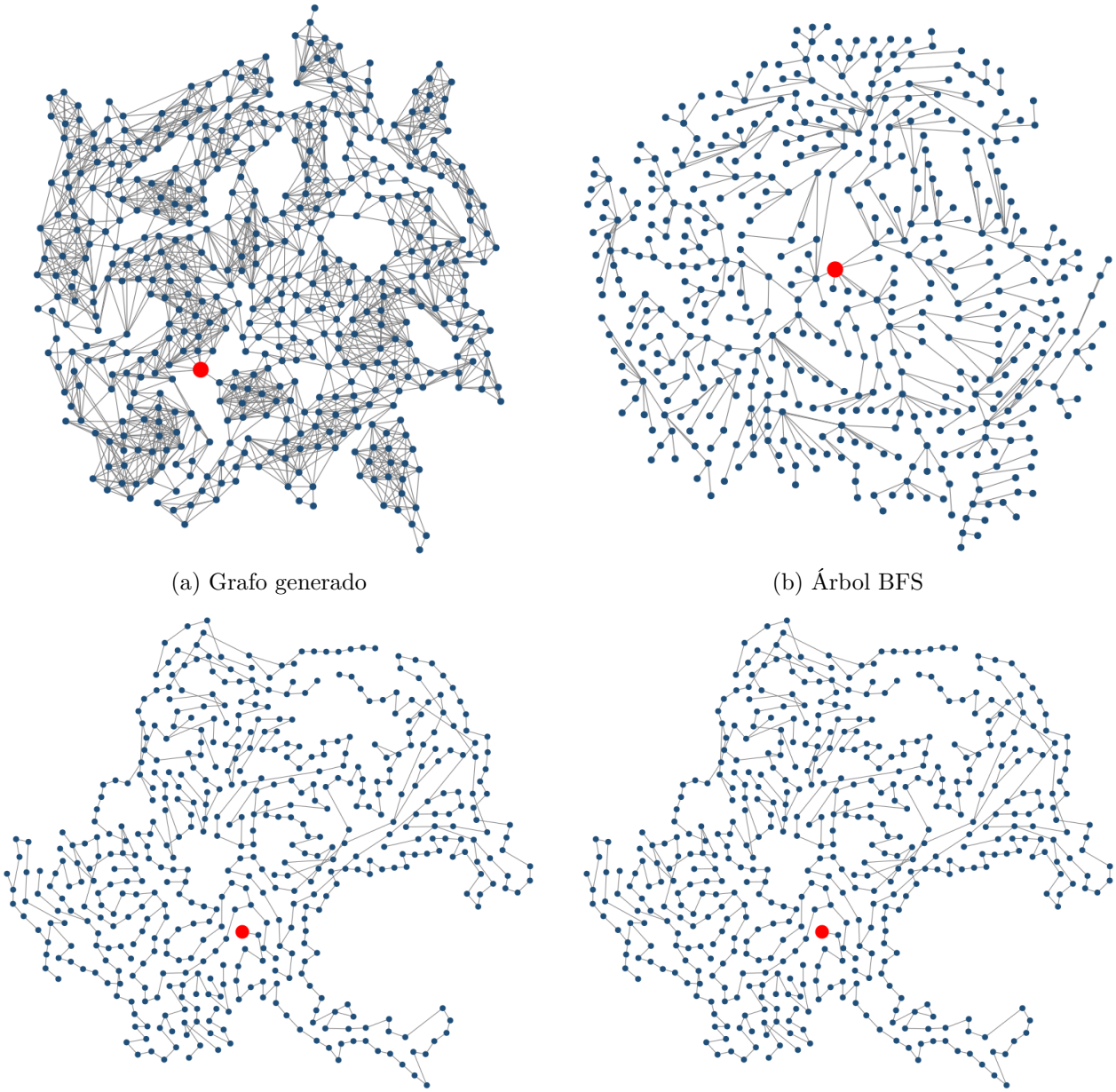
(a) Grafo generado

(b) Árbol BFS

(c) Árbol DFS recursivo

(d) Árbol DFS iterativo

Figura 14: Gilbert con 500 nodos. El nodo fuente aparece en rojo.



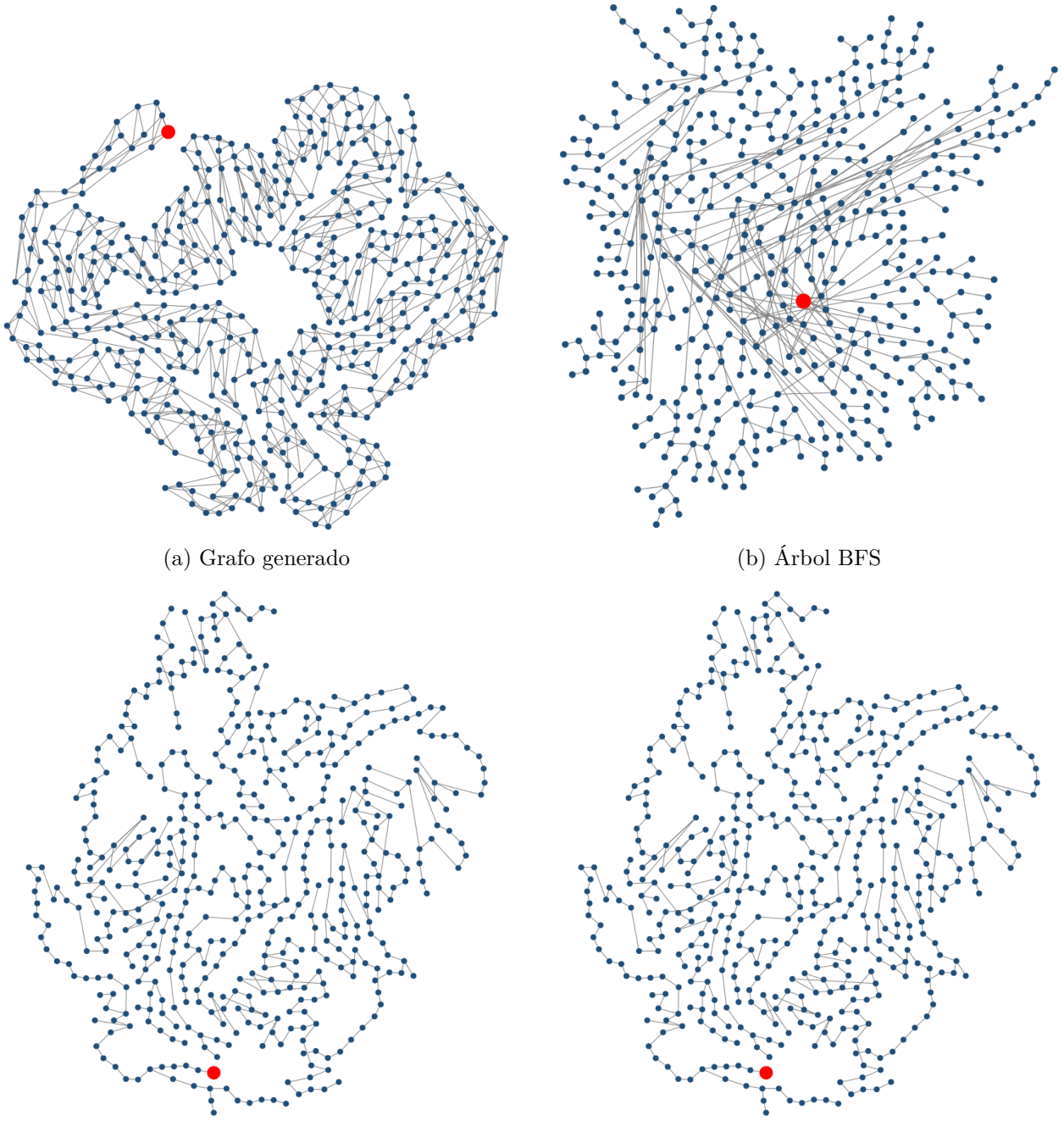
(a) Grafo generado

(b) Árbol BFS

(c) Árbol DFS recursivo

(d) Árbol DFS iterativo

Figura 15: Geográfico simple con 500 nodos. El nodo fuente aparece en rojo.



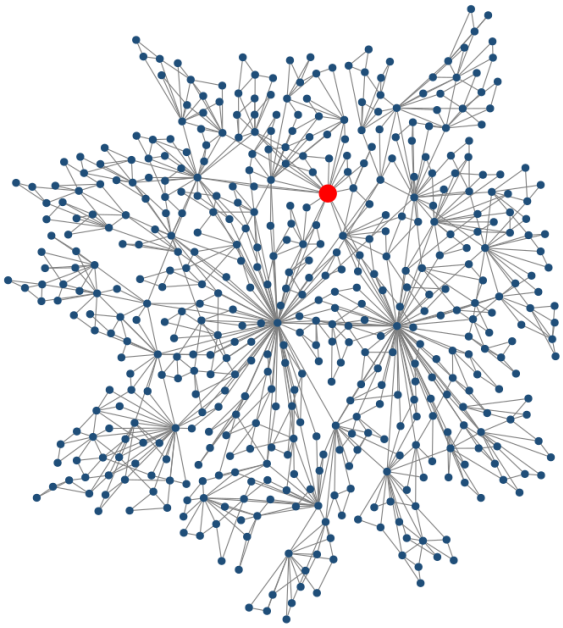
(a) Grafo generado

(b) Árbol BFS

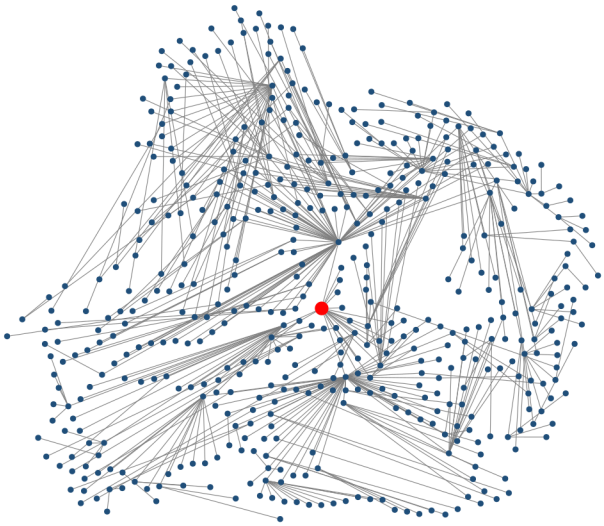
(c) Árbol DFS recursivo

(d) Árbol DFS iterativo

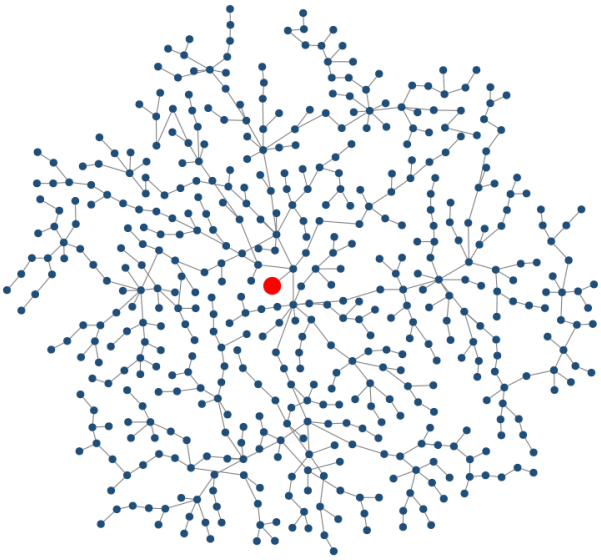
Figura 16: Barabási–Albert con 500 nodos. El nodo fuente aparece en rojo.



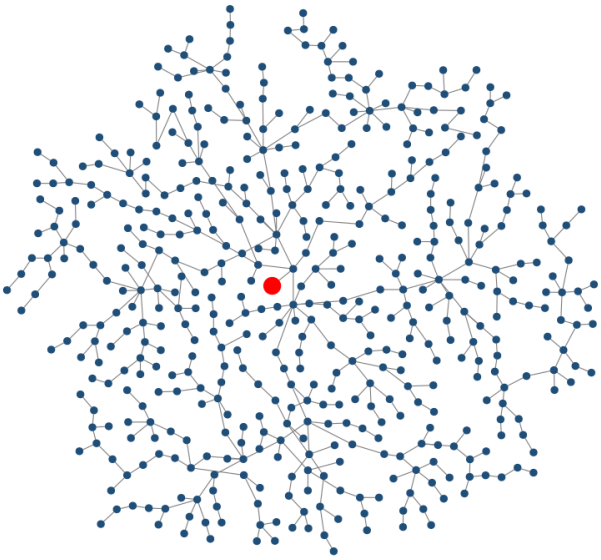
(a) Grafo generado



(b) Árbol BFS

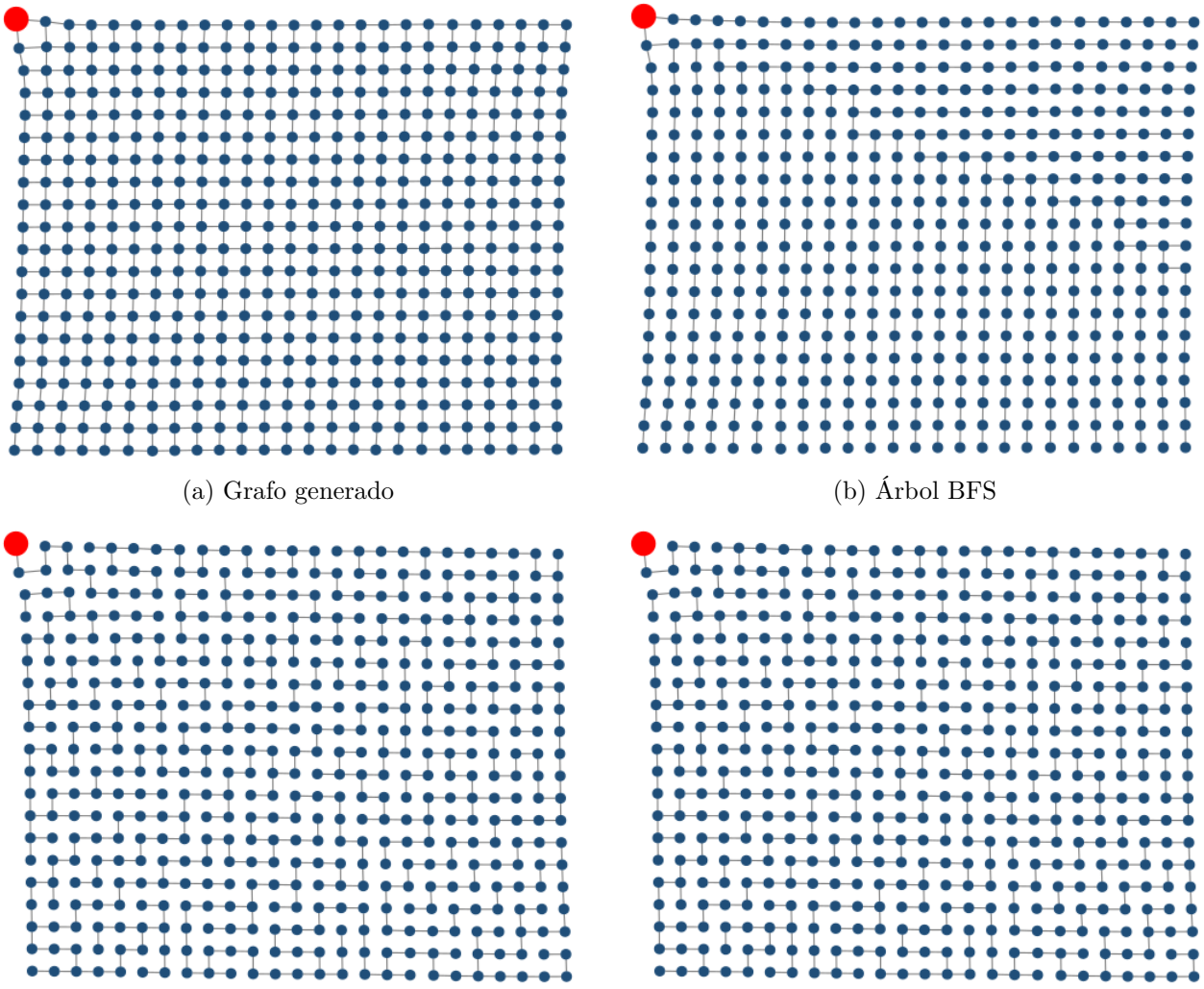


(c) Árbol DFS recursivo



(d) Árbol DFS iterativo

Figura 17: Dorogovtsev–Mendes con 500 nodos. El nodo fuente aparece en rojo.



(a) Grafo generado

(b) Árbol BFS

(c) Árbol DFS recursivo

(d) Árbol DFS iterativo

Figura 18: Malla de 20×25. El nodo fuente aparece en rojo.